МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

ОТЧЁТ

по лабораторной работе № 3

«Эмпирический анализ временной сложности алгоритмов сортировки. Часть 2: алгоритмы на основе парадигмы «разделяй и властвуй» (быстрая сортировка и сортировка слиянием)»

Дисциплина: **Теория алгоритмов**

Направление подготовки: 38.03.05 «Бизнес-информатика»

Вариант 13

Выполнила: студентка группы БИ 2-1

Кугушева Арина Вячеславовна

Проверил: _____

2026

1. Цель работы и задание варианта

Цель работы: изучить рекурсивные алгоритмы сортировки, построенные по принципу «разделяй и властвуй», экспериментально подтвердить их сложность O(n log n) и провести сравнительный анализ с квадратичными алгоритмами из лабораторной работы № 2.

Задание (вариант 13). Реализовать быструю сортировку (опорный элемент — **средний**, схема разбиения **Ломуто**) и **восходящую** (итеративную) сортировку слиянием; провести эксперименты 1–3; выполнить дополнительное задание Г1.

№	Опорный	Разбиение	Слияние	Размеры n для эксперимента 2	Доп. задание
13	С (средний)	Л (Ломуто)	В (восходящее)	10 000; 40 000; 70 000; 100 000	Г1

Г1: реализовать гибридную быструю сортировку (для подмассивов длиной ≤ M применять сортировку вставками) и подобрать оптимальное M ∈ {5, 10, 16, 32, 64} экспериментально.

Эксперименты. 1 — четыре алгоритма (пузырьком, вставками, быстрая, слиянием) на размерах варианта 13 ЛР № 2 (n = 1000…5000); 2 — быстрая, слиянием, гибридная быстрая и `sorted()` на больших n; 3 — быстрая и слиянием на данных типов R, S, V, N, D при n = 50 000. Так как в варианте опорный элемент не «первый» и не «последний», ограничение n ≤ 3000 для упорядоченных данных не требуется.

2. Описание алгоритмов, рекуррентные соотношения и оценки сложности

Парадигма «разделяй и властвуй»: задача разбивается на подзадачи, они решаются рекурсивно, результаты комбинируются. Трудоёмкость описывается рекуррентностью $T(n) = a \cdot T(n/b) + f(n)$. При $a = b = 2$ и $f(n) = \Theta(n)$ по основной теореме $T(n) = \Theta(n \log n)$: дерево рекурсии имеет $\approx \log_2 n$ уровней, на каждом уровне выполняется $\Theta(n)$ операций. Любая сортировка сравнением требует $\Omega(n \log n)$ сравнений в худшем случае (дерево решений имеет не менее $n!$ листьев).

Сортировка слиянием (восходящая). Вместо рекурсивного деления массив сразу рассматривается как n блоков длины 1; на каждом проходе соседние блоки длины w сливаются в блоки длины 2w (w = 1, 2, 4, 8, …). Проходов $\lceil \log_2 n \rceil$, каждый стоит $\Theta(n)$:

$$T(n) = \Theta(n) \cdot \lceil \log_2 n \rceil = \Theta(n \log n) \text{ — во всех случаях.}$$

Нестрогое сравнение `src[i] <= src[j]` делает сортировку устойчивой; требуется $\Theta(n)$ дополнительной памяти (буфер).

Быстрая сортировка. Средний элемент переносится в конец подмассива и служит опорным; разбиение по схеме Ломуто делит подмассив на части «≤ опорного», сам опорный элемент и «> опорного». Затем части сортируются рекурсивно. Рекурсия выполняется для *меньшей* части, а большая обрабатывается в цикле, что ограничивает глубину стека величиной O(log n). Лучший и средний случаи — $\Theta(n \log n)$; худший — $T(n) = T(n-1) + \Theta(n) = \Theta(n^2)$. Средний опорный элемент не вырождается на упорядоченных и обратно упорядоченных данных, но схема Ломуто с условием «≤» плохо работает на массивах с большим числом повторов (все равные опорному элементы попадают в одну часть) — это проверяет эксперимент 3 на данных типа D.

Гибридная быстрая сортировка (Г1). Рекурсия быстрой сортировки прекращается, когда длина подмассива не больше M; такие короткие подмассивы упорядочиваются вставками, поскольку при малых n константа сортировки вставками меньше накладных

расходов рекурсии.

Таблица 1. Теоретические оценки

Алгоритм	Лучший	Средний	Худший	Доп. память	Устойчивость
Пузырьком	Θ(n)	Θ(n²)	Θ(n²)	O(1)	Да
Вставками	Θ(n)	Θ(n²)	Θ(n²)	O(1)	Да
Слиянием	Θ(n log n)	Θ(n log n)	Θ(n log n)	Θ(n)	Да
Быстрая	Θ(n log n)	Θ(n log n)	Θ(n²)	O(log n)	Нет

Методика измерений та же, что в ЛР № 2: `time.perf_counter()`, одинаковые данные для всех алгоритмов (фиксированный seed), k = 5 повторов и медиана (для подбора M в Г1 — k = 7), проверка результата по `sorted()` вне замера, функции не изменяют исходный массив.

4. Блок-схемы алгоритмов

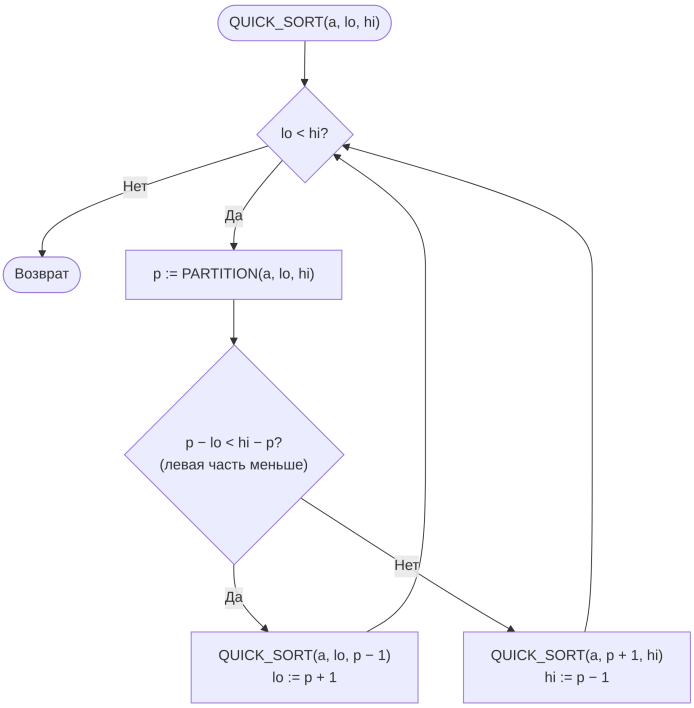


Рис. 1. Блок-схема быстрой сортировки

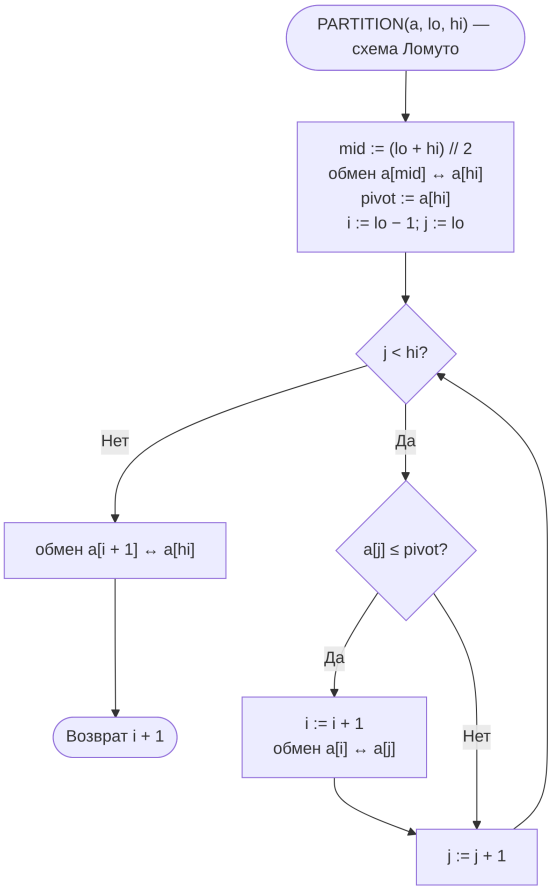


Рис. 2. Блок-схема разбиения по схеме Ломуто (средний опорный элемент)

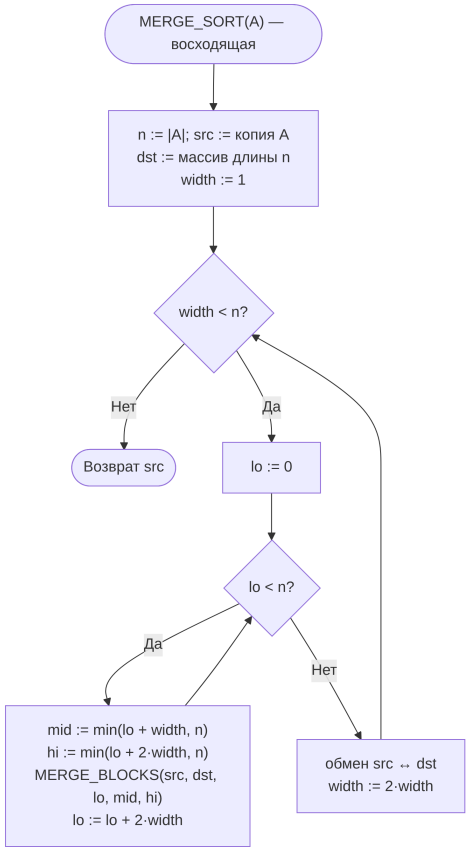


Рис. 3. Блок-схема восходящей сортировки слиянием

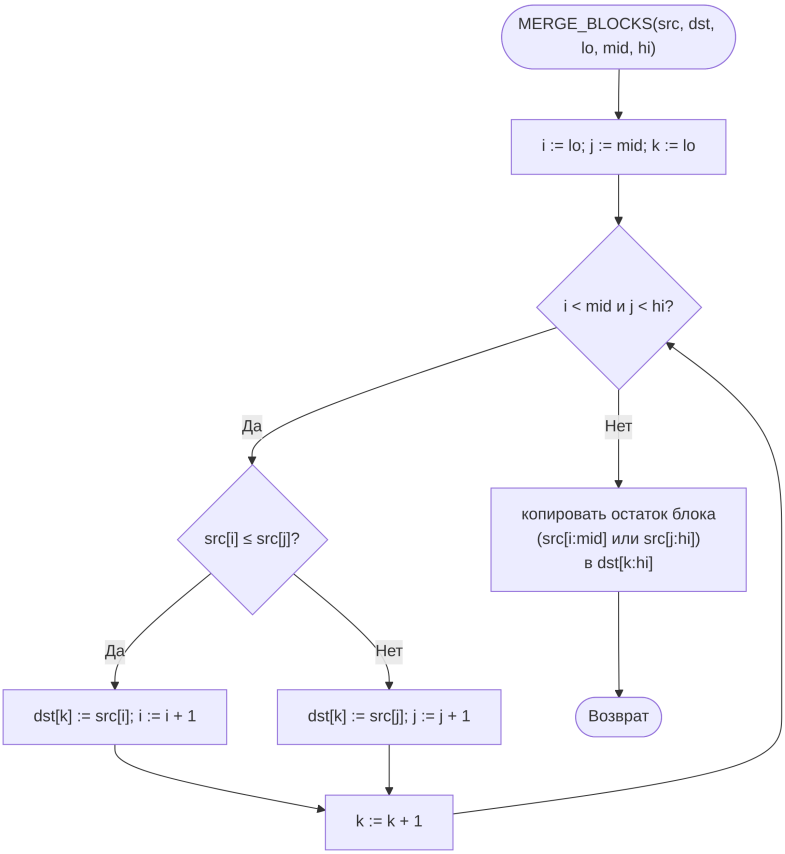


Рис. 4. Блок-схема слияния двух соседних блоков

5. Листинг программы

Файл `lr3_variant13.py` (самодостаточный модуль: функции ЛР № 2, новые алгоритмы, эксперименты и графики).

```
"""
Лабораторная работа № 3. Вариант 13.
Дисциплина: Теория алгоритмов.
Выполнила: Кугушева Арина Вячеславовна, группа БИ 2-1.

Алгоритмы «разделяй и властвуй»: быстрая сортировка и сортировка слиянием.

Исходные данные варианта 13:
    опорный элемент - средний (C);
    схема разбиения - Ломуто (Л);
    сортировка слиянием - восходящая, итеративная (В);
    размеры для эксперимента 2: 10 000; 40 000; 70 000; 100 000;
    дополнительное задание Г1 - гибридная быстрая сортировка:
    для подмассивов длиной <= M применяется сортировка вставками,
    оптимальное M выбирается из {5, 10, 16, 32, 64} экспериментально.

Эксперименты:
    1 - четыре алгоритма (пузырьком, вставками, быстрая, слиянием) на
        размерах из варианта 13 ЛР № 2 (n = 1000..5000);
    2 - быстрая, слиянием, гибридная быстрая и sorted() на больших n;
    3 - быстрая и слиянием на данных R, S, V, N, D при n = 50 000.

Файл самодостаточен (функции ЛР № 2 включены сюда же).
Запуск:  python lr3_variant13.py          (график сохраняется в lr3_plot.png)
         python lr3_variant13.py --show   (дополнительно показать график)
Полный прогон занимает около минуты (быстрая сортировка по схеме Ломуто
на данных типа D работает квадратично - это ожидаемый результат).
Требуется: matplotlib.
"""
import json
import math
import platform
import random
import statistics
import sys
import time

import matplotlib

if "--show" not in sys.argv:
    matplotlib.use("Agg")          # без окна: график только сохраняется в файл
import matplotlib.pyplot as plt

# ---------------------------- параметры варианта ----------------------------
SIZES_1 = [1000, 2000, 3000, 4000, 5000]          # размеры ЛР № 2 (вариант 13)
SIZES_2 = [10_000, 40_000, 70_000, 100_000]       # эксперимент 2 (вариант 13)
SIZES_2B = [12_500, 25_000, 50_000, 100_000]      # удвоение n для T(2n)/T(n)
N_3 = 50_000                                      # эксперимент 3
M_VALUES = [5, 10, 16, 32, 64]                    # Г1: кандидаты на порог M
N_G1 = 100_000                                    # размер массива для подбора M
REPEATS = 5
REPEATS_G1 = 7                                    # для подбора M (разница малая)
SEED = 42


# ---------------------------- генерация данных ----------------------------
def generate_data(n, kind="random", lo=0, hi=100_000, seed=SEED):
    """Массив длины n: random (R), sorted (S), reversed (V),
    nearly_sorted (N, n/20 случайных перестановок), dup (D, значения 0..10)."""
    rng = random.Random(seed + n)
    if kind == "dup":
        return [rng.randint(0, 10) for _ in range(n)]
    data = [rng.randint(lo, hi) for _ in range(n)]
    if kind == "sorted":
        data.sort()
    elif kind == "reversed":
        data.sort(reverse=True)
    elif kind == "nearly_sorted":
        data.sort()
        for _ in range(max(1, n // 20)):
            i, j = rng.randrange(n), rng.randrange(n)
            data[i], data[j] = data[j], data[i]
    return data


# ---------------------- алгоритмы ЛР № 2 (квадратичные) ----------------------
def bubble_sort(arr):
    """Сортировка пузырьком с флагом досрочного выхода."""
```

```
    a = arr.copy()
    n = len(a)
    for i in range(n - 1):
        swapped = False
        for j in range(n - 1 - i):
            if a[j] > a[j + 1]:
                a[j], a[j + 1] = a[j + 1], a[j]
                swapped = True
        if not swapped:
            break
    return a


def insertion_sort(arr):
    """Сортировка вставками."""
    a = arr.copy()
    for i in range(1, len(a)):
        key = a[i]
        j = i - 1
        while j >= 0 and a[j] > key:
            a[j + 1] = a[j]
            j -= 1
        a[j + 1] = key
    return a


# ---------------------------- быстрая сортировка ----------------------------
def quick_sort(arr):
    """Быстрая сортировка: опорный - средний, разбиение Ломуто."""
    a = arr.copy()
    _quick_sort(a, 0, len(a) - 1)
    return a


def _quick_sort(a, lo, hi):
    while lo < hi:
        p = _partition_lomuto(a, lo, hi)
        # рекурсия в меньшую часть - глубина стека O(log n)
        if p - lo < hi - p:
            _quick_sort(a, lo, p - 1)
            lo = p + 1
        else:
            _quick_sort(a, p + 1, hi)
            hi = p - 1


def _partition_lomuto(a, lo, hi):
    """Разбиение Ломуто; опорный (средний) элемент переносится в конец."""
    mid = (lo + hi) // 2
    a[mid], a[hi] = a[hi], a[mid]
    pivot = a[hi]
    i = lo - 1
    for j in range(lo, hi):
        if a[j] <= pivot:
            i += 1
            a[i], a[j] = a[j], a[i]
    a[i + 1], a[hi] = a[hi], a[i + 1]
    return i + 1                    # итоговая позиция опорного элемента


# ---------------------- Г1: гибридная быстрая сортировка ----------------------
def hybrid_quick_sort(arr, m=16):
    """Быстрая сортировка; подмассивы длиной <= m сортируются вставками."""
    a = arr.copy()
    _hybrid(a, 0, len(a) - 1, m)
    return a


def _hybrid(a, lo, hi, m):
    while hi - lo + 1 > m:
        p = _partition_lomuto(a, lo, hi)
        if p - lo < hi - p:
            _hybrid(a, lo, p - 1, m)
            lo = p + 1
        else:
            _hybrid(a, p + 1, hi, m)
            hi = p - 1
    _insertion_range(a, lo, hi)     # короткий подмассив - вставками


def _insertion_range(a, lo, hi):
    """Сортировка вставками подмассива a[lo..hi] на месте."""
    for i in range(lo + 1, hi + 1):
        key = a[i]
        j = i - 1
```

```
        while j >= lo and a[j] > key:
            a[j + 1] = a[j]
            j -= 1
        a[j + 1] = key


# --------------------- сортировка слиянием (восходящая) -----------------------
def merge_sort(arr):
    """Восходящая (итеративная) сортировка слиянием: блоки 1, 2, 4, 8, ..."""
    n = len(arr)
    src = arr.copy()
    dst = [0] * n
    width = 1
    while width < n:
        for lo in range(0, n, 2 * width):
            mid = min(lo + width, n)
            hi = min(lo + 2 * width, n)
            i, j, k = lo, mid, lo
            while i < mid and j < hi:
                if src[i] <= src[j]:         # «<=» обеспечивает устойчивость
                    dst[k] = src[i]
                    i += 1
                else:
                    dst[k] = src[j]
                    j += 1
                k += 1
            if i < mid:                      # остаток левого блока
                dst[k:hi] = src[i:mid]
            elif j < hi:                     # остаток правого блока
                dst[k:hi] = src[j:hi]
        src, dst = dst, src                  # результат слияния становится входом
        width *= 2
    return src


# ------------------------------ замер времени --------------------------------
def measure(sort_func, data, repeats=REPEATS):
    """Медианное время работы sort_func на данных data, с (perf_counter)."""
    expected = sorted(data)
    times = []
    for _ in range(repeats):
        start = time.perf_counter()
        result = sort_func(data)
        times.append(time.perf_counter() - start)
        assert result == expected, f"{sort_func.__name__}: ошибка сортировки"
    return statistics.median(times)


def run_table(algorithms, sizes, kind="random", repeats=REPEATS, scale=1.0):
    """Время каждого алгоритма для каждого n; печатает таблицу.
    Результаты возвращаются в секундах; scale=1e3 - печать в миллисекундах."""
    res = {name: [] for name in algorithms}
    print(f"{'n':>8}" + "".join(f"{name:>14}" for name in algorithms))
    for n in sizes:
        data = generate_data(n, kind)
        row = f"{n:>8}"
        for name, func in algorithms.items():
            t = measure(func, data, repeats)
            res[name].append(t)
            row += f"{t * scale:>14.4f}"
        print(row, flush=True)
    return res


def system_info():
    """Характеристики вычислительной системы (для отчёта)."""
    cpu = platform.processor() or platform.machine()
    try:
        with open("/proc/cpuinfo", encoding="utf-8") as f:
            for line in f:
                if line.startswith("model name"):
                    cpu = line.split(":", 1)[1].strip()
                    break
    except OSError:
        pass
    return (f"ОС: {platform.system()} {platform.release()}; процессор: {cpu}; "
            f"Python {platform.python_version()}")


def theory_ratio(n1, n2):
    """Теоретическое T(n2)/T(n1) для Θ(n log n)."""
    return (n2 * math.log2(n2)) / (n1 * math.log2(n1))


# ---------------------------------- main -------------------------------------
```

```
def main():
    out = {}
    print("Лабораторная работа № 3, вариант 13")
    print(system_info())
    print("Опорный: средний; разбиение: Ломуто; слияние: восходящее; "
          f"k = {REPEATS}\n")

    # --- Г1: подбор порога M гибридной сортировки (случайные данные)
    print(f"Г1. Подбор M, случайные данные, n = {N_G1}, время, с")
    data = generate_data(N_G1, "random")
    g1 = {"M=1 (без вставок)": measure(quick_sort, data, REPEATS_G1)}
    for m in M_VALUES:
        g1[f"M={m}"] = measure(lambda a, m=m: hybrid_quick_sort(a, m), data,
                               REPEATS_G1)
    for name, t in g1.items():
        print(f"{name:>20}{t:>10.4f}")
    best_m = min(M_VALUES, key=lambda m: g1[f"M={m}"])
    print(f"Оптимальное M = {best_m}\n")
    out["g1"] = g1
    out["best_m"] = best_m

    def hybrid(a):
        return hybrid_quick_sort(a, best_m)
    hybrid.__name__ = "hybrid_quick_sort"

    # --- Эксперимент 1: четыре алгоритма
    algs1 = {"Пузырьком": bubble_sort, "Вставками": insertion_sort,
             "Быстрая": quick_sort, "Слиянием": merge_sort}
    print("Эксперимент 1а. Четыре алгоритма, СЛУЧАЙНЫЕ данные, с")
    res1r = run_table(algs1, SIZES_1, "random")
    print("\nЭксперимент 1б. Четыре алгоритма, УПОРЯДОЧЕННЫЕ данные (тип S "
          "варианта 13 ЛР № 2), мс")
    res1s = run_table(algs1, SIZES_1, "sorted", scale=1e3)
    out["exp1_random"] = res1r
    out["exp1_sorted"] = res1s

    # --- Эксперимент 2: O(n log n) на больших размерах
    algs2 = {"Быстрая": quick_sort, "Слиянием": merge_sort,
             f"Гибрид M={best_m}": hybrid, "sorted()": sorted}
    print("\nЭксперимент 2. Большие массивы, СЛУЧАЙНЫЕ данные, с")
    res2 = run_table(algs2, SIZES_2, "random")
    out["exp2"] = res2
    print("\nОтношение времён (слиянием / быстрая):")
    for n, tq, tm in zip(SIZES_2, res2["Быстрая"], res2["Слиянием"]):
        print(f"{n:>8}{tm / tq:>8.2f}")
    print("\nОтношения T(n2)/T(n1) для соседних размеров эксперимента 2")
    print(f"{'Переход':>20}{'Быстрая':>10}{'Слиянием':>10}"
          f"{'Теор.nlogn':>12}{'Теор.n^2':>10}")
    ratios2 = []
    for i in range(1, len(SIZES_2)):
        n1, n2 = SIZES_2[i - 1], SIZES_2[i]
        rq = res2["Быстрая"][i] / res2["Быстрая"][i - 1]
        rm = res2["Слиянием"][i] / res2["Слиянием"][i - 1]
        th, th2 = theory_ratio(n1, n2), (n2 / n1) ** 2
        ratios2.append([n1, n2, rq, rm, th, th2])
        print(f"{str(n1) + '->' + str(n2):>20}{rq:>10.2f}{rm:>10.2f}"
              f"{th:>12.2f}{th2:>10.2f}")
    out["ratios2"] = ratios2

    print("\nЭксперимент 2б. Удвоение n, СЛУЧАЙНЫЕ данные, с")
    res2b = run_table({"Быстрая": quick_sort, "Слиянием": merge_sort},
                      SIZES_2B, "random")
    print("\nT(2n)/T(n) при удвоении")
    print(f"{'Переход':>20}{'Быстрая':>10}{'Слиянием':>10}{'Теор.nlogn':>12}")
    ratios2b = []
    for i in range(1, len(SIZES_2B)):
        n1, n2 = SIZES_2B[i - 1], SIZES_2B[i]
        rq = res2b["Быстрая"][i] / res2b["Быстрая"][i - 1]
        rm = res2b["Слиянием"][i] / res2b["Слиянием"][i - 1]
        ratios2b.append([n1, n2, rq, rm, theory_ratio(n1, n2)])
        print(f"{str(n1) + '->' + str(n2):>20}{rq:>10.2f}{rm:>10.2f}"
              f"{theory_ratio(n1, n2):>12.2f}")
    out["exp2b"] = res2b
    out["ratios2b"] = ratios2b

    # --- Эксперимент 3: структура данных
    kinds = [("random", "R"), ("sorted", "S"), ("reversed", "V"),
             ("nearly_sorted", "N"), ("dup", "D")]
    print(f"\nЭксперимент 3. Влияние структуры данных, n = {N_3}, с")
    print(f"{'Тип':>6}{'Быстрая':>12}{'Слиянием':>12}{'Слиянием/Быстрая':>20}")
    exp3 = {}
    for kind, code in kinds:
        data = generate_data(N_3, kind)
        tq = measure(quick_sort, data)
        tm = measure(merge_sort, data)
```

```
        exp3[code] = [tq, tm]
        print(f"{code:>6}{tq:>12.4f}{tm:>12.4f}{tm / tq:>20.2f}", flush=True)
    out["exp3"] = exp3

    # --- Сводная таблица (n = 5000, случайные данные)
    i5 = SIZES_1.index(5000)
    t_bubble = res1r["Пузырьком"][i5]
    print("\nСводная таблица, n = 5000, случайные данные")
    print(f"{'Алгоритм':>12}{'Время, с':>12}{'Ускорение':>12}")
    for name in algs1:
        t = res1r[name][i5]
        print(f"{name:>12}{t:>12.4f}{t_bubble / t:>12.1f}")

    # --- Графики
    fig, axes = plt.subplots(2, 2, figsize=(12, 9))
    for name, t in res1r.items():
        axes[0, 0].plot(SIZES_1, t, marker="o", label=name)
    axes[0, 0].set_yscale("log")
    axes[0, 0].set_title("Эксперимент 1 (случайные данные, лог. шкала)")
    for name, t in res2.items():
        axes[0, 1].plot(SIZES_2, t, marker="o", label=name)
    axes[0, 1].set_title("Эксперимент 2 (большие массивы)")
    for ax in axes[0]:
        ax.set_xlabel("Размер массива n")
        ax.set_ylabel("Время, с")
        ax.grid(True)
        ax.legend()
    codes = [c for _, c in kinds]
    xs = range(len(codes))
    axes[1, 0].bar([x - 0.2 for x in xs], [exp3[c][0] for c in codes], 0.4,
                   label="Быстрая")
    axes[1, 0].bar([x + 0.2 for x in xs], [exp3[c][1] for c in codes], 0.4,
                   label="Слиянием")
    axes[1, 0].set_xticks(list(xs))
    axes[1, 0].set_xticklabels(codes)
    axes[1, 0].set_yscale("log")
    axes[1, 0].set_title(f"Эксперимент 3 (n = {N_3}, лог. шкала)")
    axes[1, 0].set_xlabel("Тип данных")
    axes[1, 0].set_ylabel("Время, с")
    axes[1, 0].legend()
    axes[1, 0].grid(True, axis="y")
    axes[1, 1].plot([1] + M_VALUES, list(g1.values()), marker="o")
    axes[1, 1].set_xscale("log")
    axes[1, 1].set_title(f"Г1: подбор M (n = {N_G1})")
    axes[1, 1].set_xlabel("Порог M (M = 1 — без вставок)")
    axes[1, 1].set_ylabel("Время, с")
    axes[1, 1].grid(True)
    fig.tight_layout()
    fig.savefig("lr3_plot.png", dpi=150)
    print("\nГрафик сохранён в файл lr3_plot.png")

    if "--json" in sys.argv:                    # служебный вывод для отчёта
        with open(sys.argv[sys.argv.index("--json") + 1], "w", encoding="utf-8") as f:
            json.dump(out, f, ensure_ascii=False, indent=1)
    if "--show" in sys.argv:
        plt.show()


if __name__ == "__main__":
    main()
```

6. Характеристики вычислительной системы

Параметр	Значение
Процессор	Intel(R) Xeon(R) Processor @ 2.30GHz, 2 логических ядра
Оперативная память	8 ГБ
Операционная система	Linux (Ubuntu 24.04)
Интерпретатор	Python 3.13.18
Таймер	time.perf_counter()

Абсолютные значения времени зависят от системы; соотношения между алгоритмами и отношения $T(n_2)/T(n_1)$ должны сохраниться. Программа при запуске выводит характеристики системы автоматически.

7. Результаты экспериментов

Таблица 2. Эксперимент 1а — четыре алгоритма, случайные данные, с

n	Пузырьком	Вставками	Быстрая	Слиянием
1 000	0,0262	0,0175	0,0006	0,0008
2 000	0,1055	0,0504	0,0013	0,0018
3 000	0,2402	0,1159	0,0021	0,0028
4 000	0,4497	0,2011	0,0030	0,0038
5 000	0,6836	0,3228	0,0042	0,0050

Таблица 3. Эксперимент 1б — четыре алгоритма, упорядоченные данные (тип S варианта 13 ЛР № 2), мс

n	Пузырьком	Вставками	Быстрая	Слиянием
1 000	0,0349	0,0617	0,4122	0,4953
2 000	0,0600	0,1113	0,9173	1,0979
3 000	0,0986	0,1649	1,5482	1,8015
4 000	0,1441	0,2265	2,0319	2,3923
5 000	0,1687	0,3053	2,6323	3,2035

Таблица 4. Эксперимент 2 — алгоритмы O(n log n) на больших массивах (случайные данные), с

n	Быстрая	Слиянием	Гибрид (M = 16)	`sorted()`	Слиянием / быстрая
10 000	0,0087	0,0109	0,0080	0,0010	1,25
40 000	0,0395	0,0525	0,0363	0,0049	1,33
70 000	0,0740	0,0996	0,0693	0,0095	1,35
100 000	0,1138	0,1469	0,1063	0,0151	1,29

Таблица 5. Проверка роста: $T(n_2)/T(n_1)$ для соседних размеров эксперимента 2

Переход	Быстрая	Слиянием	Теория O(n log n)	Теория O(n^2)
10 000 → 40 000	4,52	4,80	≈ 4,60	16,00
40 000 → 70 000	1,87	1,90	≈ 1,84	3,06
70 000 → 100 000	1,54	1,47	≈ 1,47	2,04

Таблица 6. Эксперимент 2б — удвоение n (случайные данные): время, с, и T(2n)/T(n)

n	Быстрая, с	Слиянием, с	Переход	Быстрая	Слиянием	Теория
12 500	0,0106	0,0141	—	—	—	—
25 000	0,0226	0,0303	12 500 → 25 000	2,13	2,16	≈ 2,15
50 000	0,0526	0,0699	25 000 → 50 000	2,33	2,30	≈ 2,14
100 000	0,1098	0,1447	50 000 → 100 000	2,09	2,07	≈ 2,13

Таблица 7. Эксперимент 3 — влияние структуры данных (n = 50 000), с

Тип данных	Быстрая	Слиянием	Слиянием / быстрая
Случайные (R)	0,0521	0,0668	1,28
Упорядоченные (S)	0,0350	0,0456	1,30
Обратно упорядоченные (V)	0,0390	0,0537	1,38
Почти упорядоченные (N)	0,0521	0,0632	1,21
С повторами (D)	5,4269	0,0539	0,01

Таблица 8 (Г1). Подбор порога M гибридной сортировки (n = 100 000, случайные данные, k = 7), с

M	1 (без вставок)	5	10	16	32	64
Время, с	0,1109	0,1079	0,1072	0,1035	0,1109	0,1337

Минимум достигнут при M = 16: 0,1035 с против 0,1109 с без вставок (выигрыш ≈ 7 %).

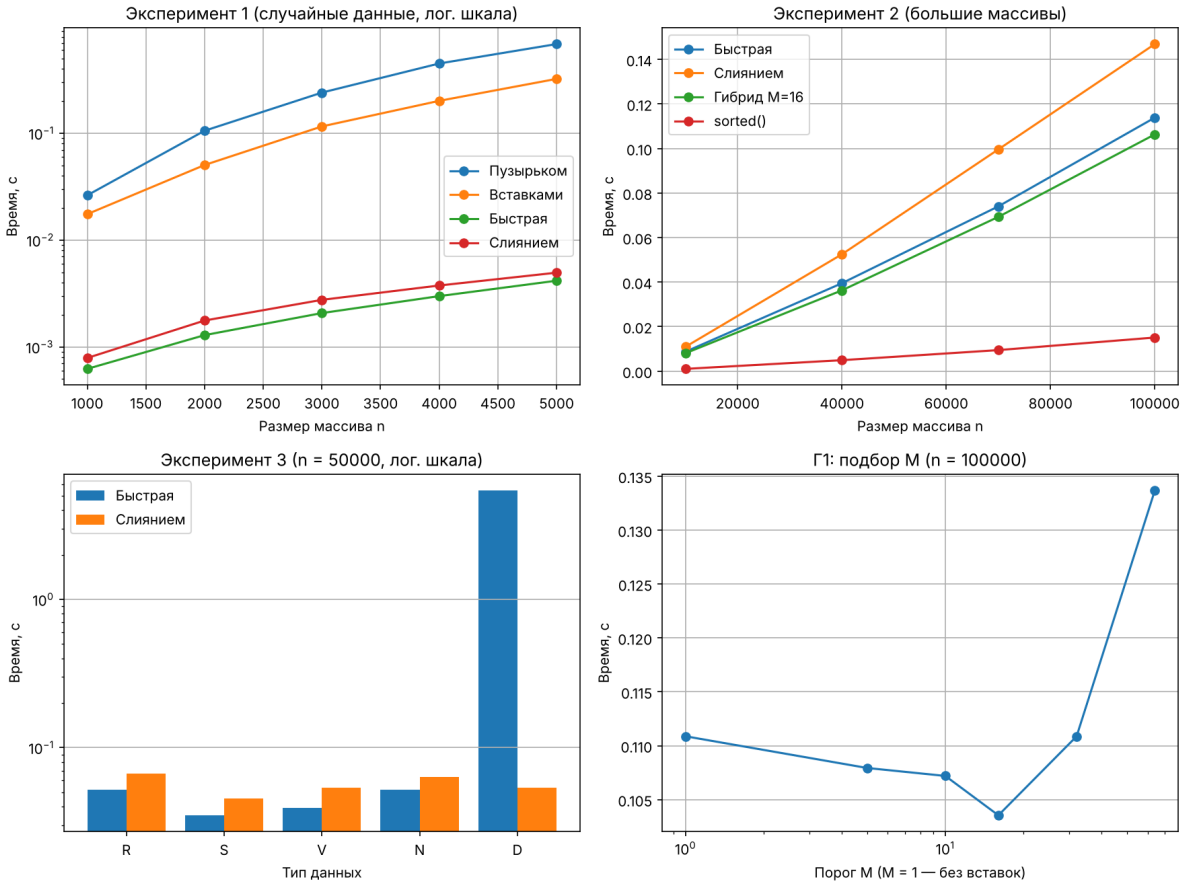


Рис. 5. Графики экспериментов: 1 (лог. шкала), 2, 3 (лог. шкала) и подбор M (Г1)

8. Сводная сравнительная таблица

Таблица 9. Итоговое сравнение алгоритмов (n = 5000, случайные данные; рост — T(4000)/T(2000), теория для Θ(n²) — 4, для Θ(n log n) ≈ 2,18)

Алгоритм	Теор. сложность (средняя)	Время, с	Ускорение относительно пузырька	Эксп. рост T(4000)/T(2000)	Память	Устойчивость
Пузырьком	Θ(n²)	0,6836	1	4,26	O(1)	Да
Вставками	Θ(n²)	0,3228	2,1	3,99	O(1)	Да
Слиянием	Θ(n log n)	0,0050	138	2,13	Θ(n)	Да
Быстрая	Θ(n log n)	0,0042	164	2,32	O(log n)	Нет

9. Выводы и рекомендации по выбору алгоритма

1. **Классы сложности различаются принципиально.** На случайных данных при n = 5000 быстрая сортировка быстрее пузырька в 164 раз, сортировка слиянием — в 138 раз; относительно сортировки вставками — в 77 и 65 раз. На логарифмической шкале (рис. 5) кривые квадратичных алгоритмов растут значительно круче, и разрыв увеличивается с ростом n.
2. **Адаптивность квадратичных алгоритмов.** На упорядоченных данных (табл. 3) картина обратная: пузырёк с флагом и вставки работают за Θ(n) и при n = 5000 занимают 0,17 и 0,31 мс, тогда как неадаптивные быстрая сортировка и слияние — 2,6 и 3,2 мс, то есть примерно в 9–19 раз медленнее. Вывод: для почти упорядоченных данных сортировка вставками остаётся хорошим выбором.

3. **Подтверждение оценки Θ(n log n).** При удвоении n время быстрой сортировки и слияния возрастает в 2,09–2,33 раза (табл. 6), теория даёт ≈ 2,13–2,15, квадратичный рост дал бы 4. Для размеров варианта 13 (табл. 5) измеренные отношения $T(n_2)/T(n_1)$ близки к теоретическим для n log n (например, 10 000 → 40 000: 4,52 и 4,80 при теории 4,60) и существенно ниже квадратичных (16). Отклонения до 10 % объясняются кэш-памятью и накладными расходами интерпретатора.
4. **Быстрая сортировка быстрее слияния** в 1,25–1,35 раза (табл. 4). Она работает на месте и не создаёт новых блоков, тогда как слияние на каждом проходе перекладывает все n элементов в буфер. Разрыв меньше, чем у рекурсивной версии слияния из образца задания (1,6–1,7 раза); вероятная причина — восходящая версия использует один заранее выделенный буфер и не создаёт срезов списков.
5. **Влияние структуры данных (эксперимент 3).** Со средним опорным элементом быстрая сортировка не деградирует на упорядоченных и обратно упорядоченных данных: 0,0350 и 0,0390 с против 0,0521 с на случайных — разбиения получаются сбалансированными (для первого или последнего опорного эти данные были бы худшим случаем $\Theta(n^2)$). Однако на данных типа D (значения 0…10) схема Ломуто с условием «≤» вырождается: 5,43 с, то есть в 104 раз медленнее, чем на случайных данных, и в 101 раз медленнее сортировки слиянием (0,0539 с). Все элементы, равные опорному, попадают в одну часть, каждая из 11 групп равных значений (≈ 4500 элементов) обрабатывается за квадратичное время. Сортировка слиянием от повторов не зависит. Для таких данных нужно трёхпутевое разбиение или схема Хоара.
6. **Гибридизация (Г1).** Оптимальный порог M = 16 дал выигрыш около 7 % на n = 100 000 (табл. 8); значения M = 5…16 различаются в пределах погрешности измерения, а M = 64 уже хуже чистой быстрой сортировки (0,1337 с против 0,1109 с): квадратичная сортировка вставками на длинных подмассивах съедает выигрыш. В эксперименте 2 гибридная версия быстрее обычной на 6–8 %.
7. **Встроенная сортировка.** Функция `sorted()` быстрее собственной быстрой сортировки в 7,5–8,5 раза, а собственной сортировки слиянием — в 9,7–10,7 раза (реализация на C, гибридный адаптивный алгоритм). В прикладных информационных системах следует использовать стандартные библиотечные средства сортировки; собственные реализации оправданы лишь в учебных целях или при особых требованиях.
8. **Рекомендации по выбору.** Нужна устойчивость (многоуровневая сортировка отчётов) или гарантированное время Θ(n log n) независимо от данных — сортировка слиянием. Важны экономия памяти и средняя скорость — быстрая сортировка, но с защитой от повторов (трёхпутевое разбиение) и от упорядоченных данных (средний, случайный или медиана трёх опорный элемент), желательно в гибридном варианте с вставками для коротких подмассивов. Малые массивы (n < 50) и почти упорядоченные данные — сортировка вставками. Для больших объёмов, не помещающихся в память, нужна внешняя сортировка (на основе слияния).

10. Список литературы

1. Кормен Т., Лейзерсон Ч., Ривест Р., Штайн К. Алгоритмы: построение и анализ. — 3-е изд. — М.: Вильямс, 2013. — Гл. 2, 4, 7, 8.
2. Седжвик Р., Уэйн К. Алгоритмы на Java. — 4-е изд. — М.: Вильямс, 2013. — Гл. 2.
3. Hoare C. A. R. Quicksort // The Computer Journal. — 1962. — Vol. 5, No. 1. — P. 10–16.
4. Скиена С. Алгоритмы. Руководство по разработке. — 3-е изд. — СПб.: БХВ-Петербург, 2022.

5. Кнут Д. Э. Искусство программирования. Т. 3. Сортировка и поиск. — 2-е изд. — М.: Вильямс, 2017.